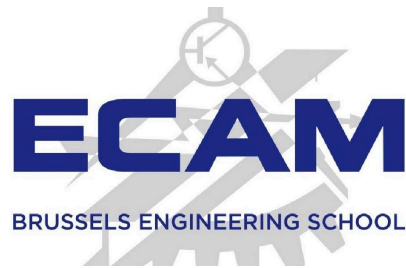


Floris Vassily, 4MIN
21245

Schell Nicolas, 3BE
21242

Yildirim Arifcan, 4MIN
20144



Lab2: Configuration d'une DMZ

Prom. de l'Alma 50, Woluwe-Saint-Lambert.

Supervisor : LORIAU Nicolas

11th october 2025

Table des matières

1 Introduction	3
2 Laboratoire	3
2.1 Configuration des machines virtuelles (VirtualBox)	3
2.1.1 Création d'un réseau interne dédié à la DMZ	4
2.2 Installation du serveur Ubuntu	4
2.3 Configuration de pfSense	5
2.3.1 Création de l'interface DMZ	5
2.3.2 Configuration du serveur DHCP	6
2.3.3 Attribution d'une adresse IP fixe au serveur Ubuntu	6
2.3.4 Création des règles de firewall	7
2.3.5 Vérification de la connectivité Internet	8
2.3.6 Vérification du blocage de communication au LAN	9
2.4 Configuration du port forwarding	9
2.4.1 Redirection du port HTTP (80) et HTTPS (443)	9
2.4.2 Vérification des règles NAT	10
2.5 Déploiement du serveur web avec Docker et Nginx	10
2.5.1 Installation de Docker	10
2.5.2 Création du conteneur Nginx	10
2.5.3 Vérification du fonctionnement de Nginx	11
2.6 Mise en place d'un reverse proxy avec Apache	12
2.6.1 Installation d'Apache	12
2.6.2 Activation des modules nécessaires	12
2.6.3 Certificat autosigné et sécurisation HTTPS	12
2.6.4 Configuration du reverse proxy	13
2.7 Mise en place d'un DNS autoritaire avec BIND	13
2.7.1 Installation de BIND sur pfSense	13
2.7.2 Configuration générale de BIND	14
2.7.3 Création d'une view DNS	14
2.7.4 Création de la zone DNS	15
2.7.5 Autorisation des requêtes DNS depuis Internet	15
2.7.6 Vérification du DNS autoritaire	16
2.8 Validation finale et problème rencontré avec le domaine .dev	16
3 Conclusion	17
4 Sources :	18

1 Introduction

L'objectif de ce laboratoire est de mettre en place une DMZ (**demilitarized zone**) afin d'isoler un serveur web du **réseau interne**. Cette architecture permet d'exposer uniquement les services nécessaires vers l'extérieur, tout en limitant l'accès direct aux machines internes.

Dans ce labo, nous avons configuré une infrastructure composée de deux éléments principaux :

- une machine virtuelle pfSense, utilisée comme pare-feu et routeur afin de gérer les flux entre le réseau interne, la DMZ et l'extérieur ;
- un serveur Ubuntu placé dans la DMZ, destiné à héberger un service web accessible en HTTP et en HTTPS.

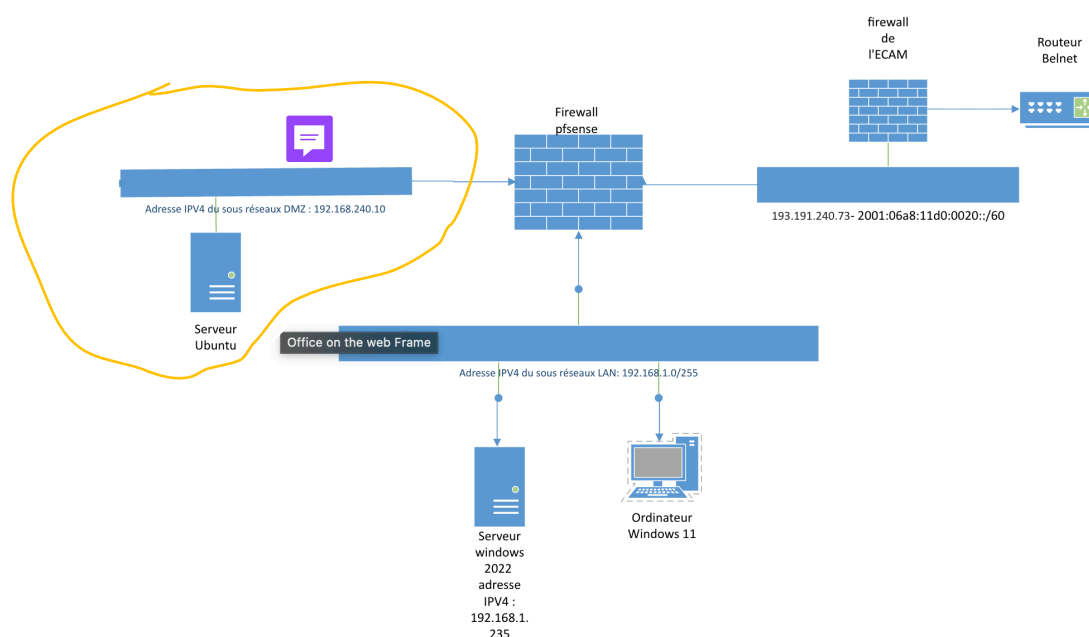


Figure 1: Architecture réseau du laboratoire

L'objectif est donc de comprendre le fonctionnement d'une DMZ, de configurer pfSense pour segmenter correctement les communications réseau, puis de déployer un serveur web accessible depuis l'extérieur tout en conservant la sécurité des autres zones du réseau.

Ce rapport présente les différentes étapes de configuration, depuis la création des machines virtuelles jusqu'à la mise en service du serveur web dans la DMZ.

2 Laboratoire

2.1 Configuration des machines virtuelles (VirtualBox)

Avant de commencer la configuration réseau, nous avons créé la machine virtuelle destinée au serveur web dans VirtualBox. La machine a été préparée avec les caractéristiques suivantes :

- 2 Go de mémoire vive ;
- 1 processeur ;
- 25 Gio de stockage.
- Ubuntu server 24.04 LTS

L'espace disque de 25 Gio peut sembler important pour un serveur web relativement léger. Cependant, avec la version d'Ubuntu Server utilisée lors du labo, une capacité plus faible a provoqué des problèmes durant l'installation. Nous avons donc choisi une taille de disque plus confortable afin d'éviter les erreurs et de garantir une installation stable.

2.1.1 Création d'un réseau interne dédié à la DMZ

La configuration la plus importante sur VirtualBox pour ces machines virtuelles concerne leur interface réseau. Nous avons ajouté un adaptateur configuré en réseau interne, nommé **DMZ**, afin de permettre la communication entre pfSense et le serveur Ubuntu.

Cette séparation réseau permet d'isoler le serveur web du reste du réseau local et de le placer dans une zone contrôlée par le pare-feu.

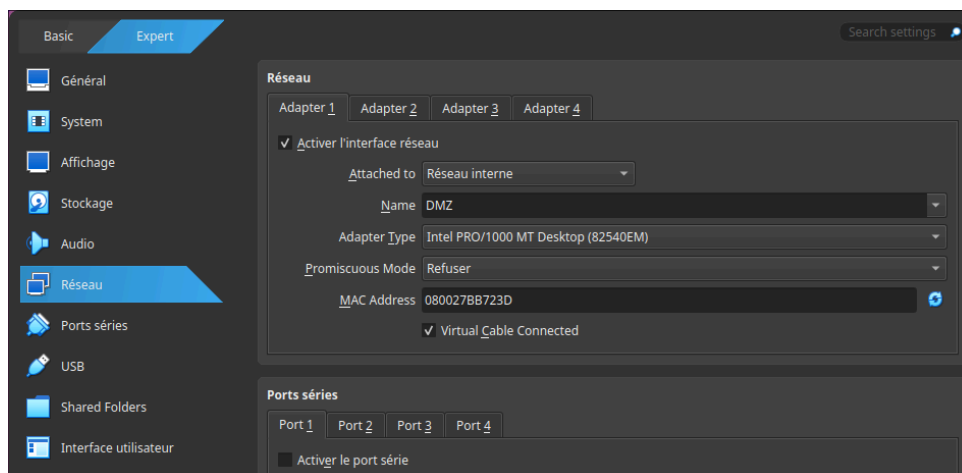


Figure 2: Configuration du réseau interne dans le serveur Ubuntu

Une fois cette configuration effectuée, le serveur Ubuntu peut communiquer avec pfSense via le réseau interne **DMZ**. Nous avons ensuite lancé l'installation du système d'exploitation en suivant l'assistant d'installation affiché ci-dessous.

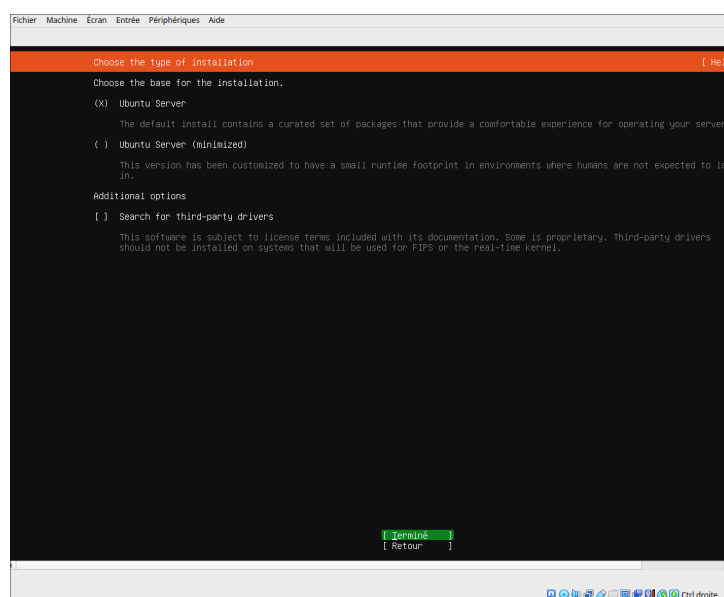


Figure 3: Choix de la version du serveur Ubuntu

2.2 Installation du serveur Ubuntu

Après avoir sélectionné l'image ISO d'Ubuntu Server, nous avons suivi les différentes étapes de l'assistant d'installation. A ce stade, l'objectif est simplement d'obtenir une machine fonctionnelle, capable de rejoindre le réseau DMZ et de recevoir une adresse IP correcte via la configuration prévue sur pfSense.

Une fois l'installation terminée, nous avons vérifié que le serveur disposait bien d'une adresse IP appartenant au réseau défini pour la DMZ. Cette étape est essentielle avant de poursuivre la configuration des services réseau et du pare-feu.

Une fois l'image ISO lancée, nous avons poursuivi l'assistant d'installation d'Ubuntu Server jusqu'à la création du compte utilisateur local.

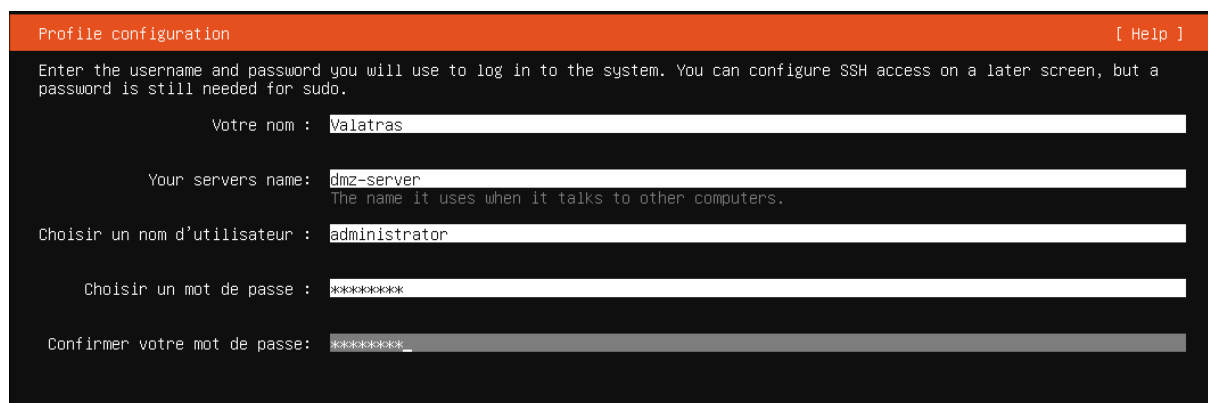


Figure 4: Configuration du compte utilisateur du serveur Ubuntu

Cette étape termine l'installation du système et permet ensuite d'accéder au serveur pour vérifier sa connectivité réseau ainsi que le bon fonctionnement des services qui seront installés par la suite.

Après le redémarrage, le serveur Ubuntu est prêt à être intégré dans la DMZ et à recevoir sa configuration réseau via pfSense.

2.3 Configuration de pfSense

Une fois le serveur Ubuntu installé, nous avons configuré pfSense afin de créer et gérer le réseau DMZ. Cette partie est essentielle, car pfSense joue le rôle de passerelle entre le réseau interne, la DMZ et Internet. 

2.3.1 Création de l'interface DMZ

Dans pfSense, nous avons ajouté une nouvelle interface réseau dédiée à la DMZ. Cette interface a été configurée en adresse IPv4 statique avec l'adresse **192.168.240.1 (/24)**.

Cette adresse représente la passerelle du réseau DMZ et permet au serveur Ubuntu de communiquer avec pfSense.

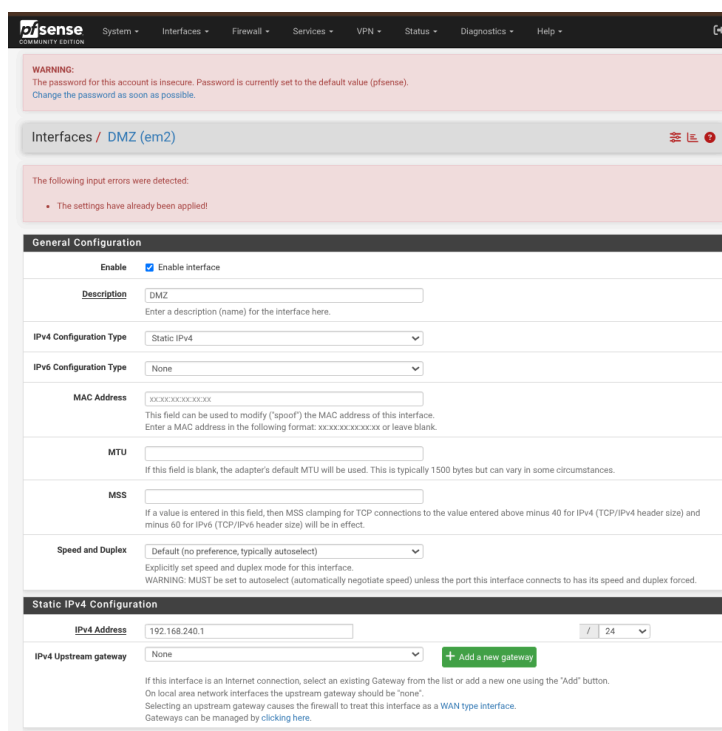


Figure 5: Création de l'interface DMZ sur pfSense

2.3.2 Configuration du serveur DHCP

Afin de faciliter l'attribution des adresses IP dans la DMZ, nous avons activé le service DHCP sur cette nouvelle interface.

La plage d'adresses configurée pour le serveur DHCP est la suivante :

- 192.168.240.20 à 192.168.240.230

Cette configuration permet d'attribuer automatiquement une adresse IP aux machines qui rejoignent le réseau DMZ, tout en gardant une plage réservée pour d'autres usages.

The screenshot shows the pfSense web interface for configuring the DHCP Server on the DMZ interface. At the top, a green message states 'The changes have been applied successfully.' Below this, a yellow warning banner indicates that ISC DHCP has reached end-of-life and will be removed in a future version of pfSense, suggesting a switch to the DHCP backend. The interface has tabs for WAN, LAN, and DMZ, with DMZ selected. The 'General Settings' section includes options for the DHCP Backend (ISC DHCP), Enable (checked), BOOTP (unchecked), Deny Unknown Clients (set to 'Allow all clients'), Ignore Denied Clients (unchecked), and Ignore Client Identifiers (unchecked). The 'Primary Address Pool' section shows the Subnet as 192.168.240.0/24, the Subnet Range as 192.168.240.1 - 192.168.240.254, and the Address Pool Range from 192.168.240.20 to 192.168.240.230. An 'Add Address Pool' button is visible at the bottom.

Figure 6: Configuration du service DHCP sur l'interface DMZ

2.3.3 Attribution d'une adresse IP fixe au serveur Ubuntu

Pour le serveur Ubuntu hébergé dans la DMZ, nous avons configuré un bail DHCP statique basé sur son adresse MAC. Cela permet d'associer de manière permanente l'adresse IPv4 192.168.240.10 à cette machine.

Cette méthode garantit que le serveur conserve toujours la même adresse IP, ce qui est important pour la configuration du pare-feu, du port forwarding et des futurs services web.

Services / DHCP Server / DMZ / Static Mapping / Edit

ISC DHCP has reached end-of-life and will be removed in a future version of pfSense. Visit [System > Advanced > Networking](#) to switch DHCP backend.

Static DHCP Mapping

DHCP Backend	ISC DHCP
MAC Address	08:00:27:bc:e8:3d
MAC address of the client to match (6 hex octets separated by colons).	
Client Identifier	DMZ_Ubuntu_Server
An optional identifier to match based on the value sent by the client (RFC 2132).	
IP Address	192.168.240.10
IPv4 address to assign this client.	
Address must be outside of any defined pools. If no IPv4 address is given, one will be dynamically allocated from a pool. The same IP address may be assigned to multiple mappings.	

Figure 7: Allocation d'une adresse IPv4 fixe pour le serveur Ubuntu

2.3.4 Création des règles de firewall

Une fois l'interface DMZ et le DHCP configurés, nous avons ajouté les règles de firewall nécessaires afin de contrôler les échanges réseau entre la DMZ et le reste de l'infrastructure.

L'objectif principal de cette règle est d'autoriser l'accès depuis la DMZ vers Internet, tout en empêchant l'accès direct au réseau LAN.

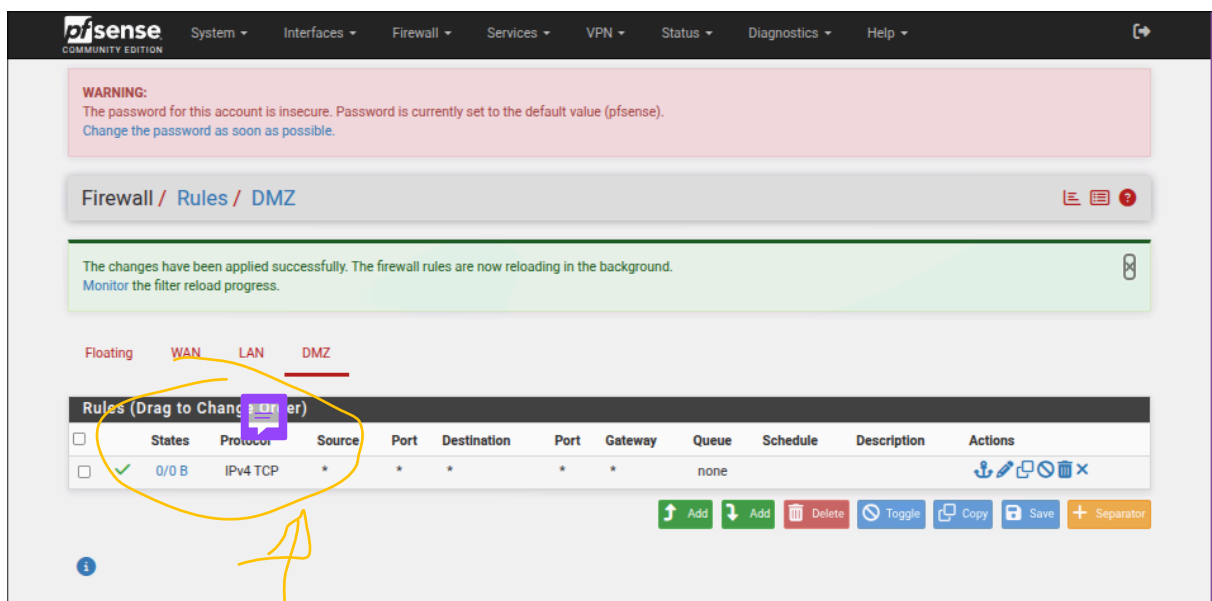


Figure 8: Règles de firewall appliquées à l'interface DMZ

La règle a été configurée avec les paramètres suivants :

- Action : 'Pass'
- Interface : 'DMZ'
- Destination : 'LAN Address'

avec l'option d'inversion activée.

Cette configuration autorise le trafic sortant depuis la DMZ, tout en **excluant le réseau LAN** comme destination. Ainsi, le serveur Ubuntu peut accéder à Internet, mais ne peut pas communiquer librement avec les machines internes du réseau local.

Firewall / Rules / Edit

Edit Firewall Rule

Action
 Choose what to do with packets that match the criteria specified below.
 Hint: the difference between block and reject is that with reject, a packet (TCP RST or ICMP port unreachable for UDP) is returned to the sender, whereas with block the packet is dropped silently. In either case, the original packet is discarded.

Disabled ☐ Disable this rule
 Set this option to disable this rule without removing it from the list.

Interface
 Choose the interface from which packets must come to match this rule.

Address Family
 Select the Internet Protocol version this rule applies to.

Protocol
 Choose which IP protocol this rule should match.

Source

Source ☐ Invert match /

Destination

Destination ☒ Invert match /

Extra Options

Log ☐ Log packets that are handled by this rule
 Hint: the firewall has limited local log space. Don't turn on logging for everything. If doing a lot of logging, consider using a remote syslog server (see the Status: System Logs: Settings page).

Description
 A description may be entered here for administrative reference. A maximum of 52 characters will be used in the ruleset label and displayed in the firewall log.

Advanced Options

Rule Information

Tracking ID	1774521776
Created	3/26/26 10:42:56 by admin@151.236.158.22 (Local Database)
Updated	3/26/26 10:45:12 by admin@151.236.158.22 (Local Database)

Figure 9: Configuration de la règle de firewall sur l'interface DMZ

2.3.5 Vérification de la connectivité Internet

Après la configuration de pfSense, nous avons vérifié que le serveur Ubuntu pouvait bien accéder à Internet. Pour cela, nous avons effectué un test de connectivité à l'aide d'une commande `ping`.

```
administrator@dmz-server:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=3.61 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=4.40 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=3.42 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=3.33 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=3.41 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=3.39 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=117 time=3.54 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=117 time=3.43 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=117 time=3.44 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=117 time=3.35 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=117 time=3.44 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=117 time=3.45 ms
^C
--- 8.8.8.8 ping statistics ---
12 packets transmitted, 12 received, 0% packet loss, time 11011ms
rtt min/avg/max/mdev = 3.325/3.517/4.399/0.276 ms
administrator@dmz-server:~$
```

Figure 10: Test de ping du serveur Ubuntu vers Internet en passant par pfSense

Ce test confirme que la configuration de la DMZ est fonctionnelle : le serveur Ubuntu reçoit bien une adresse IP valide, passe par pfSense pour joindre l'extérieur, et reste séparé du réseau LAN.

2.3.6 Vérification du blocage de communication au LAN

Il est important d'inclure dans nos tests une tentative de communication de notre machine de la DMZ avec une machine du LAN pour vérifier que la communication est bien bloquée par notre règle de firewall. Nous avons donc fait une requête de ping depuis notre serveur ubuntu (DMZ) vers une machine windows client de notre LAN et nous avons ainsi pu confirmer que la communication était bien bloquée. L'enregistrement d'une illustration de cette étape a été malheureusement omise lors de notre laboratoire.



2.4 Configuration du port forwarding

Une fois la connectivité de la DMZ validée, nous avons configuré le **port forwarding** sur pfSense afin de rendre le serveur web accessible depuis l'extérieur.

L'objectif est de rediriger les connexions arrivant sur l'interface WAN de pfSense vers le serveur Ubuntu situé dans la DMZ. Cela permet aux utilisateurs externes d'accéder au site web hébergé sur le serveur sans exposer directement l'ensemble du réseau interne.

2.4.1 Redirection du port HTTP (80) et HTTPS (443)

La première règle NAT concerne le port **80**, utilisé pour le protocole HTTP.

Cette règle redirige automatiquement les requêtes arrivant depuis Internet vers le serveur Ubuntu situé à l'adresse **192.168.240.10**.

Les paramètres principaux sont les suivants :

```
Adresse du serveur DMZ : '192.168.240.10'
Source : 'ouverte (connexions extérieures autorisées)'
Destination : 'interface WAN de pfSense'
Destination Port Range : 'HTTP'
Redirect Target Port : 'HTTP'
Redirect Target IP : '192.168.240.10'
```



Figure 11: Configuration du port forwarding pour HTTP

Nous avons ensuite créé la même configuration pour le port `443`, utilisé pour le protocole HTTPS.

Cette seconde règle permet de sécuriser l'accès au site web via TLS/SSL et sera utilisée plus tard avec Apache et le reverse proxy.

2.4.2 Vérification des règles NAT

Après la création des deux règles, pfSense génère automatiquement les règles de pare-feu associées afin d'autoriser le trafic entrant sur ces ports.

Nous pouvons ainsi vérifier la présence des redirections HTTP et HTTPS dans la table NAT.

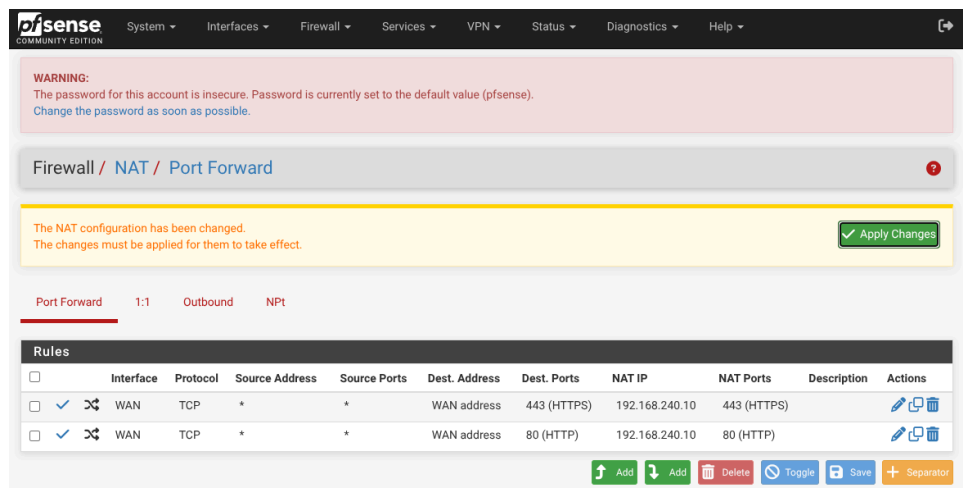


Figure 12: Règles de port forwarding créées sur pfSense

2.5 Déploiement du serveur web avec Docker et Nginx

Une fois l'accès réseau configuré, nous pouvons déployer un premier serveur web sur la machine Ubuntu à l'aide de Docker et de Nginx.

Cette approche permet d'isoler le service web dans un conteneur léger et facilement reproductible.

2.5.1 Installation de Docker

Avant de déployer le serveur web, nous avons installé Docker sur le serveur Ubuntu afin de pouvoir exécuter Nginx dans un conteneur isolé.

Docker permet de simplifier le déploiement des services en encapsulant l'application et sa configuration dans une image reproductible.

Les commandes utilisées sont les suivantes :

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker administrator
```

Une reconnexion de la session utilisateur est nécessaire pour appliquer les nouveaux droits.

2.5.2 Création du conteneur Nginx

Nous avons créé un dossier de travail contenant :

- un `Dockerfile`
- un fichier de configuration `nginx.conf`
- un dossier `html` contenant la page web à servir

Le `Dockerfile` utilisé repose sur l'image officielle légère `nginx:alpine`.

Le fichier de configuration `default.conf` utilisé pour Nginx est volontairement minimal.

Ce dernier permet simplement à Nginx d'écouter sur le port `80` et de servir un potentiel contenu html.

```
administrator@dmz-server:~/web$ ls
default.conf Dockerfile html
administrator@dmz-server:~/web$ cat default.conf
server {
    listen: 80

    location / {
        root /home/administrator/web/html;
        index index.html;
    }
}
```

Figure 13: configuration de nginx

Le conteneur est ensuite construit puis exécuté avec les commandes suivantes :

```
docker build -t nginx .
docker run -d -p 80:80 --name nginx-test nginx
```

Cette configuration permet de rendre le site accessible localement sur le port `80` de la machine Ubuntu. Elle suffit pour valider le fonctionnement du serveur web dans la DMZ avant la mise en place du reverse proxy HTTPS avec Apache.

2.5.3 Vérification du fonctionnement de Nginx

Nous avons vérifié le bon fonctionnement du serveur web en testant l'accès depuis le réseau local (Windows Client sur le LAN), puis depuis l'extérieur via les règles de port forwarding configurées précédemment.

Ces tests confirment que le trafic HTTP traverse correctement pfSense jusqu'au conteneur Nginx.

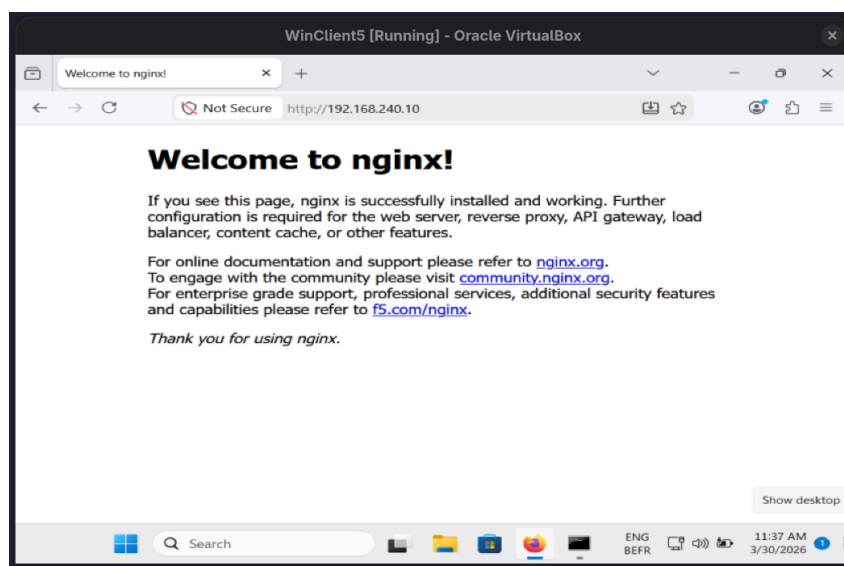


Figure 14: Test d'accès au serveur Nginx depuis le réseau local

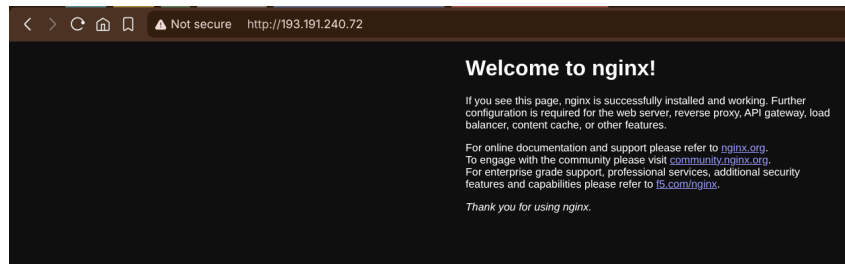


Figure 15: Test d'accès au serveur Nginx depuis l'extérieur de notre réseau

2.6 Mise en place d'un reverse proxy avec Apache

Après validation du fonctionnement de Nginx, nous avons mis en place un reverse proxy avec Apache afin de gérer les connexions HTTPS.

Il y'a plusieurs manières de mettre en place notre reverse proxy :

1. **SSL Offloading** : Le proxy de pfSense s'occupe de décrypter/encrypter en HTTPS et transmet en HTTP interne (ce qui réduit la charge du serveur ubuntu backend). Cela implique d'accepter d'avoir du trafic en clair (non-crypté) à l'intérieur de notre réseau. Le service Squid sur pfSense peut réaliser cela.
2. **Terminaison SSL** : Le proxy ne décrypte pas le trafic du tout. Il transmet simplement le stream TCP encrypté à Apache qui déchiffre le HTTPS et transmet en HTTP à Nginx.

Nous avons réalisé les 2 options mais cette deuxième option a été retenue pour la suite : Apache centralise les certificats SSL et allège Nginx.

2.6.1 Installation d'Apache

Nous avons commencé par installer Apache sur le serveur Ubuntu avec la commande suivante :

```
sudo apt install apache2 -y
sudo systemctl status apache2
```

Cela permet de vérifier que le service est correctement installé et actif.

2.6.2 Activation des modules nécessaires

Pour permettre le fonctionnement du reverse proxy HTTPS, plusieurs modules Apache doivent être activés :

le support SSL ; le module proxy ; le module proxy HTTP ; le site SSL par défaut.

Les commandes utilisées sont les suivantes :

```
sudo a2enmod ssl
sudo a2enmod proxy
sudo a2enmod proxy_http
sudo a2ensite default-ssl.conf
sudo systemctl restart apache2
```

2.6.3 Certificat autosigné et sécurisation HTTPS

Afin de permettre l'accès sécurisé au site web via HTTPS, Apache doit disposer d'un certificat numérique.

Historiquement, cette sécurisation reposait sur le protocole SSL (**Secure Sockets Layer**). Aujourd'hui, SSL est considéré comme obsolète et a été remplacé par TLS (**Transport Layer Security**), qui offre une meilleure sécurité et corrige plusieurs faiblesses des anciennes versions de SSL.

Dans la pratique, on continue souvent à parler de "certificat SSL", bien qu'il s'agisse en réalité d'un certificat utilisé avec TLS.

Pour ce laboratoire, nous avons utilisé un certificat autosigné (**self-signed certificate**), suffisant pour les tests internes même s'il n'est pas reconnu par les autorités de certification publiques.

La génération du certificat a été réalisée avec la commande suivante :

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/ssl-cert-snakeoil.key \
-out /etc/ssl/certs/ssl-cert-snakeoil.pem
```

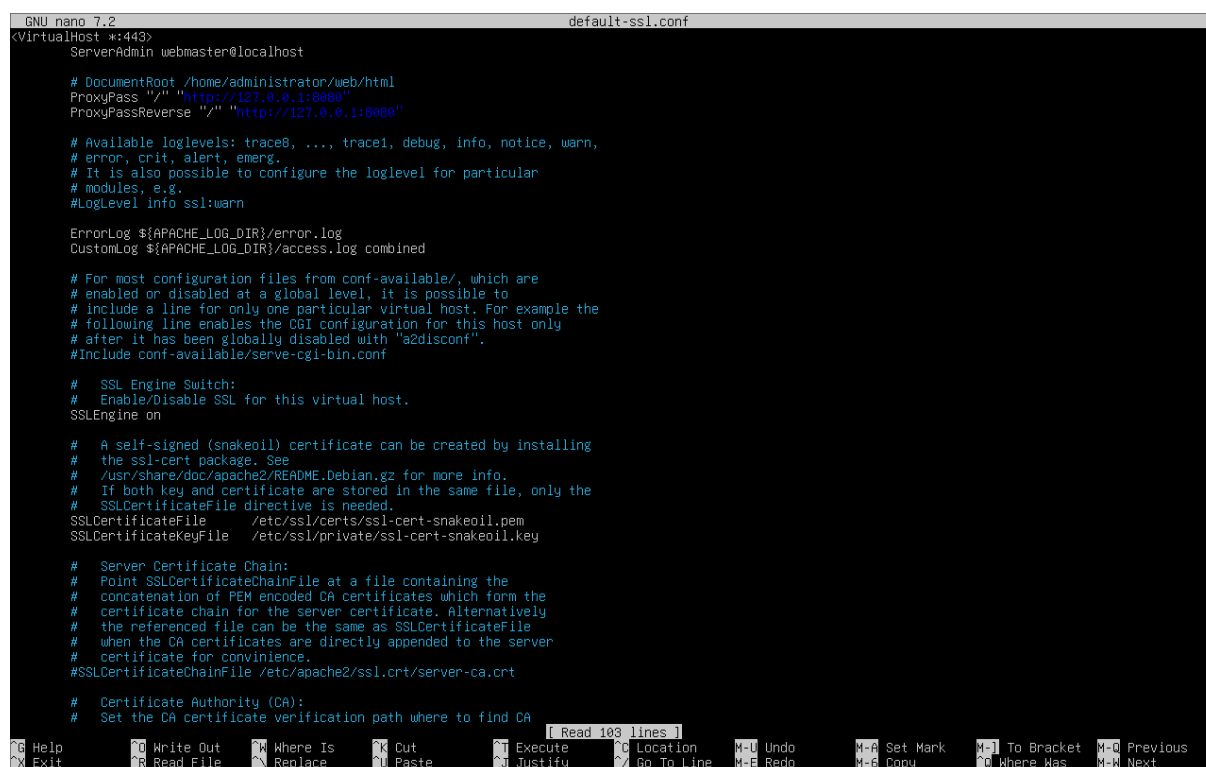
2.6.4 Configuration du reverse proxy

Nous avons ensuite modifié le fichier :

```
/etc/apache2/sites-available/default-ssl.conf
```

afin qu'Apache écoute les connexions HTTPS et redirige les requêtes vers le conteneur Nginx exécuté sur le port **8080** de la machine Ubuntu.

Cette configuration permet à Apache de jouer le rôle de point d'entrée sécurisé, tandis que Nginx reste le serveur qui fournit réellement le contenu du site.



```
GNU nano 7.2 default-ssl.conf
<VirtualHost *:443>
    ServerAdmin webmaster@localhost

    # DocumentRoot /home/administrator/web/html
    ProxyPass "/" "http://127.0.0.1:8080"
    ProxyPassReverse "/" "http://127.0.0.1:8080"

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf

    # SSL Engine Switch:
    # Enable/Disable SSL for this virtual host.
    SSLEngine on

    # A self-signed (snakeoil) certificate can be created by installing
    # the ssl-cert package. See
    # /usr/share/doc/apache2/README.Debian.gz for more info.
    # If both key and certificate are stored in the same file, only the
    # SSLCertificateFile directive is needed.
    SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem
    SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key

    # Server Certificate Chain:
    # Point SSLCertificateChainFile at a file containing the
    # concatenation of PEM encoded CA certificates which form the
    # certificate chain for the server certificate. Alternatively
    # the referenced file can be the same as SSLCertificateFile
    # when the CA certificates are directly appended to the server
    # certificate for convenience.
    #SSLCertificateChainFile /etc/apache2/ssl.crt/server-ca.crt

    # Certificate Authority (CA):
    # Set the CA certificate verification path where to find CA
```

Figure 16: Configuration d'Apache en reverse proxy HTTPS

2.7 Mise en place d'un DNS autoritaire avec BIND

Afin de rendre notre serveur web accessible via un nom de domaine plutôt qu'une simple adresse IP, nous avons mis en place un serveur DNS autoritaire sur pfSense à l'aide du package BIND.

L'objectif est de faire résoudre notre domaine directement par pfSense, en associant le nom de domaine public à l'adresse IP WAN utilisée pour accéder au serveur web hébergé dans la DMZ.

2.7.1 Installation de BIND sur pfSense

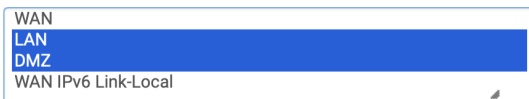
Nous avons commencé par installer le package BIND depuis le gestionnaire de paquets de pfSense (**Package Manager**).

Ce service permet à pfSense d'agir comme serveur DNS autoritaire pour notre domaine et de répondre directement aux requêtes DNS externes.

Lors de l'installation, nous avons défini les interfaces utilisées par le service DNS afin de contrôler sur quels réseaux BIND doit écouter.

The port used for responding to SSL/TLS DNS queries. It should normally be left blank unless another service needs to bind to TCP/UDP port 853.

Network Interfaces



Interface IP addresses used by the DNS Resolver for responding to queries from clients. If an interface has both IPv4 and IPv6 addresses, both are

Figure 17: Interfaces utilisées par le service DNS avec BIND

2.7.2 Configuration générale de BIND

Dans la configuration générale, nous avons désactivé le DNS Resolver sur l'interface WAN afin de conserver uniquement les interfaces LAN et DMZ.

Cela permet d'éviter les conflits entre le DNS Resolver natif de pfSense et le service BIND configuré pour servir notre zone DNS autoritaire.

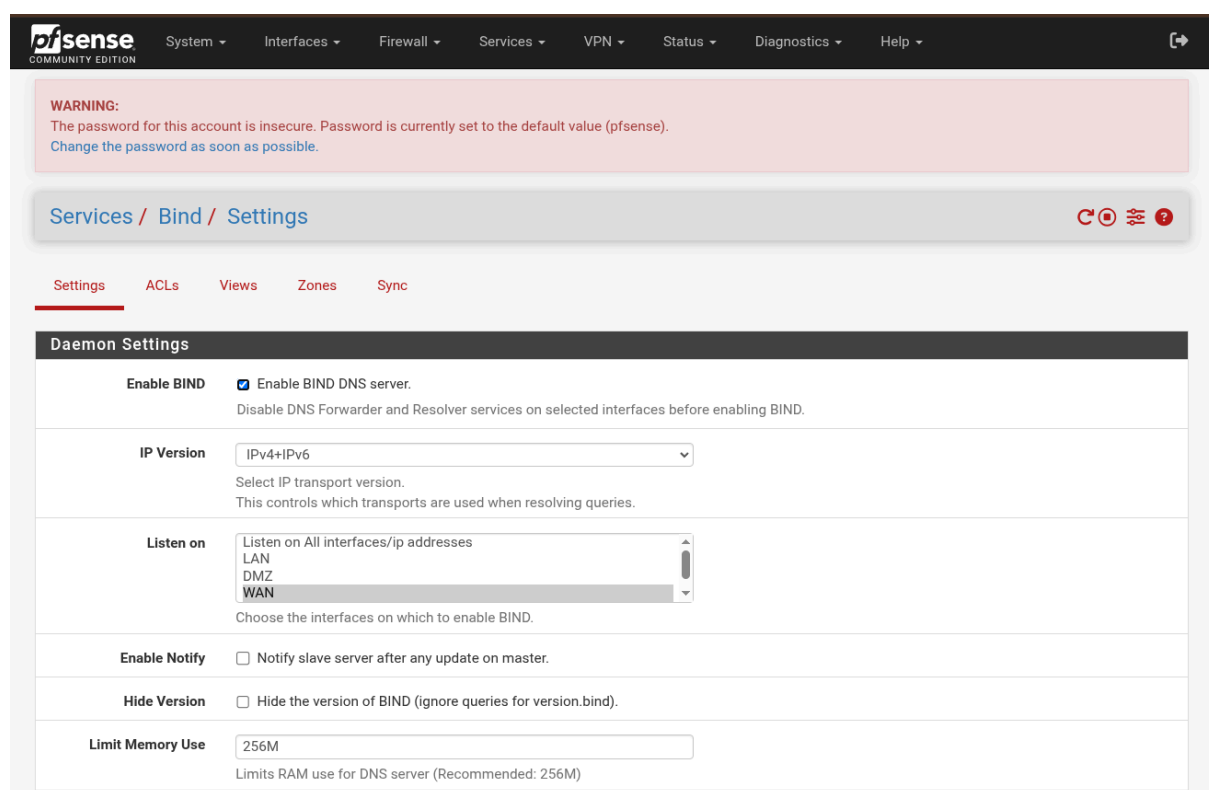


Figure 18: Configuration générale de BIND

2.7.3 Création d'une view DNS

Nous avons ensuite créé une **view** DNS afin de définir le comportement des requêtes reçues.

Les paramètres principaux sont les suivants :

- Nom : `nginx`
- Recursion : `no`
- Match-clients : `any`
- Allow-recursion : `none`

Le fait de désactiver la récursion est important pour un DNS autoritaire : le serveur répond uniquement pour les zones qu'il héberge et ne réalise pas de résolution récursive pour d'autres domaines.

Figure 19: Configuration de la view DNS

2.7.4 Création de la zone DNS

Nous avons ensuite créé la zone DNS correspondant à notre domaine.

Dans cette zone, nous avons défini les enregistrements nécessaires pour permettre la résolution du site web ainsi que du serveur de noms.

Le fichier de zone contient notamment :

- un enregistrement **SOA (Start of Authority)** ;
- un enregistrement **NS** pour le serveur de noms ;
- un enregistrement **A** pour le domaine principal ;
- un enregistrement **A** pour **www** ;
- un enregistrement **A** pour le serveur **ns** .

La configuration utilisée est la suivante :

```
$TTL 14400

$ORIGIN netlab.dev.

netlab.dev. IN SOA ns. zonemaster.netlab.dev. (
    2775113178 ; serial
    1d        ; refresh
    2h        ; retry
    4w        ; expire
    1h        ; default_ttl
)

@ IN NS ns.
@ IN A 193.191.240.73
ns IN A 193.191.240.73
www IN A 193.191.240.73
```

Cette configuration permet de faire pointer le domaine ainsi que le sous-domaine **www** vers l'adresse IP publique utilisée pour atteindre pfSense depuis Internet.

2.7.5 Autorisation des requêtes DNS depuis Internet

Afin que les requêtes DNS puissent atteindre BIND depuis l'extérieur, nous avons ajouté une règle de pare-feu sur pfSense autorisant le trafic UDP sur le port **53** depuis l'interface WAN.

Sans cette règle, les requêtes DNS externes seraient bloquées avant d'atteindre le service BIND.

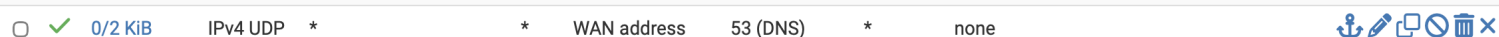


Figure 20: Règle firewall autorisant le trafic DNS UDP sur le WAN

2.7.6 Vérification du DNS autoritaire

Nous avons validé le bon fonctionnement du DNS autoritaire à l'aide de la commande `dig`.

Le premier test a été réalisé avec le domaine `netlab.dev`, ce qui a permis de confirmer que la résolution DNS répondait correctement et que le domaine pointait bien vers notre adresse publique.

```
peterschell@MacBook-Air-6 / % dig @193.191.240.73 www.netlab.dev

; <<>> DiG 9.10.6 <<>> @193.191.240.73 www.netlab.dev
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 64250
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;www.netlab.dev.                IN      A

;; ANSWER SECTION:
www.netlab.dev.                14400   IN      A      193.191.240.73

;; Query time: 27 msec
;; SERVER: 193.191.240.73#53(193.191.240.73)
;; WHEN: Thu Apr 02 09:20:13 CEST 2026
;; MSG SIZE rcvd: 59
```

Figure 21: Test du DNS autoritaire avec la commande dig

2.8 Validation finale et problème rencontré avec le domaine `.dev`

Lors de nos premiers tests HTTPS, nous utilisons un nom de domaine en `.dev`. Cependant, cette extension est soumise par défaut à la politique HSTS (**HTTP Strict Transport Security**) imposée par les navigateurs modernes.

Cela signifie que le navigateur exige automatiquement un certificat SSL valide signé par une autorité reconnue, ce qui empêche l'utilisation simple d'un certificat autosigné pour les tests.

Cette contrainte a provoqué plusieurs échecs lors de l'accès au site web, malgré une configuration correcte du reverse proxy, du port forwarding et du DNS.

Pour résoudre ce problème, nous avons remplacé le domaine `.dev` par un domaine en `.be`, qui ne force pas automatiquement HSTS. Cela nous a permis d'utiliser notre certificat autosigné et de valider correctement l'accès HTTPS.

Le test final a confirmé que le site était accessible depuis l'extérieur via HTTPS, avec la résolution DNS correcte et le reverse proxy Apache fonctionnel.

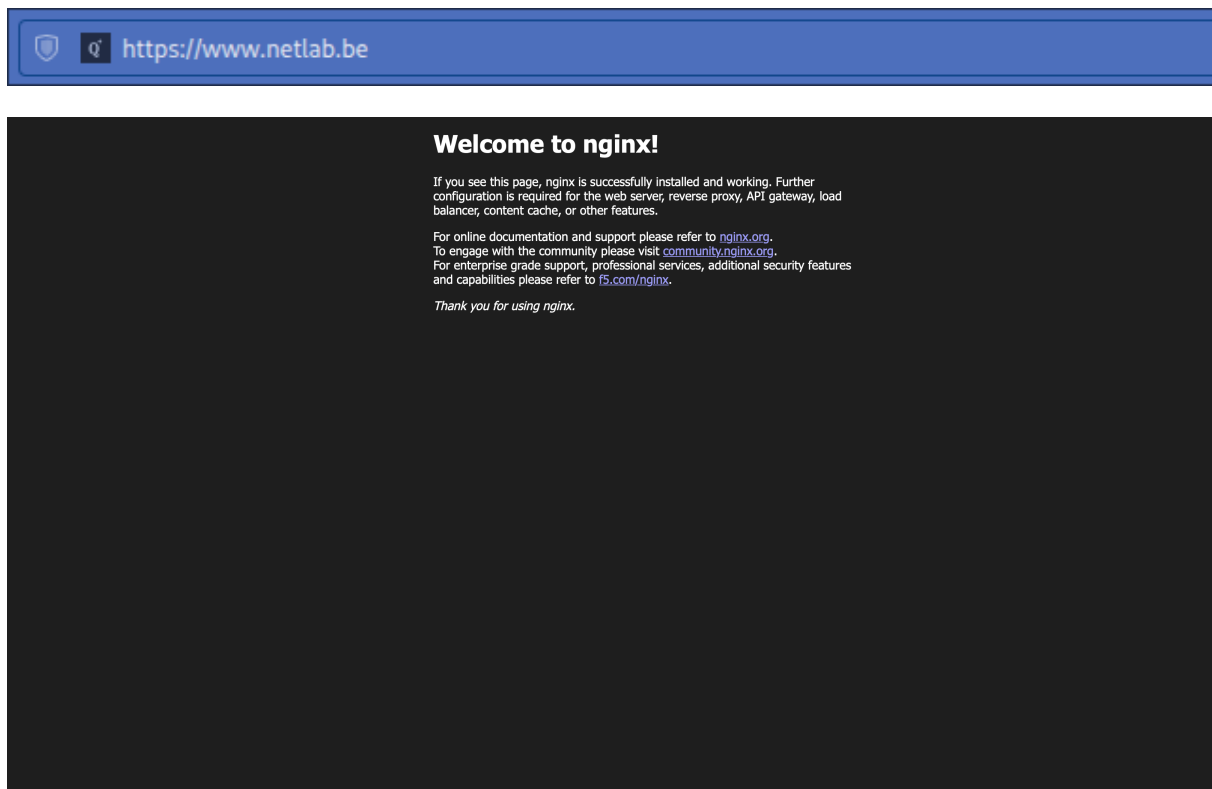


Figure 23: Validation finale de l'accès HTTPS avec le domaine netlab.be

3 Conclusion

Ce laboratoire nous a permis de mettre en place une architecture réseau complète intégrant une DMZ sécurisée autour d'un serveur web accessible depuis Internet.

Nous avons commencé par la création d'un serveur Ubuntu placé dans un réseau isolé, la zone démilitarisée. Puis nous avons configuré pfSense afin de gérer l'interface DMZ, le service DHCP, les règles de pare-feu ainsi que le port forwarding nécessaire pour publier les services HTTP et HTTPS.

Nous avons ensuite déployé un serveur web avec Docker et Nginx, puis mis en place un reverse proxy HTTPS avec Apache afin de centraliser la gestion du chiffrement SSL tout en conservant Nginx comme serveur de contenu.

Enfin, l'ajout d'un DNS autoritaire avec BIND nous a permis d'associer un nom de domaine public à notre infrastructure et de valider un accès complet depuis l'extérieur.

Ce laboratoire illustre l'importance de la segmentation réseau, de la maîtrise des règles de pare-feu et de la sécurisation des services exposés sur Internet. Il nous a également montré certains problèmes, comme la politique HSTS imposée par certains domaines tels que `.dev`.

L'ensemble de ces manipulations nous a permis d'obtenir une meilleure compréhension du rôle d'une DMZ et des mécanismes nécessaires pour publier un service de manière sécurisée dans une infrastructure réseau professionnelle.

4 Sources :

- pfSense Documentation : <https://docs.netgate.com/pfsense/en/latest/>
- ubuntu server Documentation : <https://ubuntu.com/server/docs/>
- Vbox : <https://www.virtualbox.org/manual/>
- .dev hsts requirements : <https://dodov.dev/blog/avoid-dev-domains-for-test-environments>
- create an autosigned certificate : <https://www.digitalocean.com/community/tutorials/how-to-create-a-self-signed-ssl-certificate-for-apache-in-ubuntu-22-04>